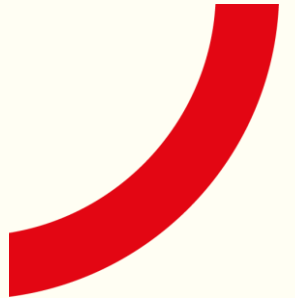


CSC1029 OO Programming

Lecture 11 – Class Data, Methods & Enumerated Types





What's to come in today's lecture?

- Class Data
- Class Methods
- Enumerated Types
- A Bank Account Example

Class Variables & Class Methods (1)

BankAccount

<pre>- name : String - acctNum : int - balance : double + BankAccount(String, int) + deposit(double):boolean + withdraw(double):boolean + statement(): String</pre>
--

```
BankAccount acct1 = new
    BankAccount("Paul", 1234) ;
```

```
BankAccount acct2 = new
    BankAccount("Fred", 4321) ;
```

```
acct2.deposit(200) ;
acct2.withdraw(50) ;
```

```
acct1.deposit(20) ;
String str=acct2.statement() ;
```

Class Variables & Class Methods (2)

BankAccount
- name : String - acctNum : int - balance : double
+ BankAccount(String, int) + deposit(double):boolean + withdraw(double):boolean + statement(): String

acct1



"Paul"
1234
159.5

Instance variables –
separate values in each
object

"Fred"

4321
234.0



acct2

Instance methods –
accessed via object
reference

Class Variables & Class Methods (3)

BankAccount
- name : String - acctNum : int - balance : double
+ BankAccount(String, int) + deposit(double):boolean + withdraw(double):boolean + statement(): String

acct1

"Paul"
1234
159.5

```
public BankAccount(String name, int number)
{
    this.name = name;
    this.acctNum = number;
    this.balance = 0.0;
}
```

What alternative to providing account number as a construction parameter?

Class Variables & Class Methods (4)

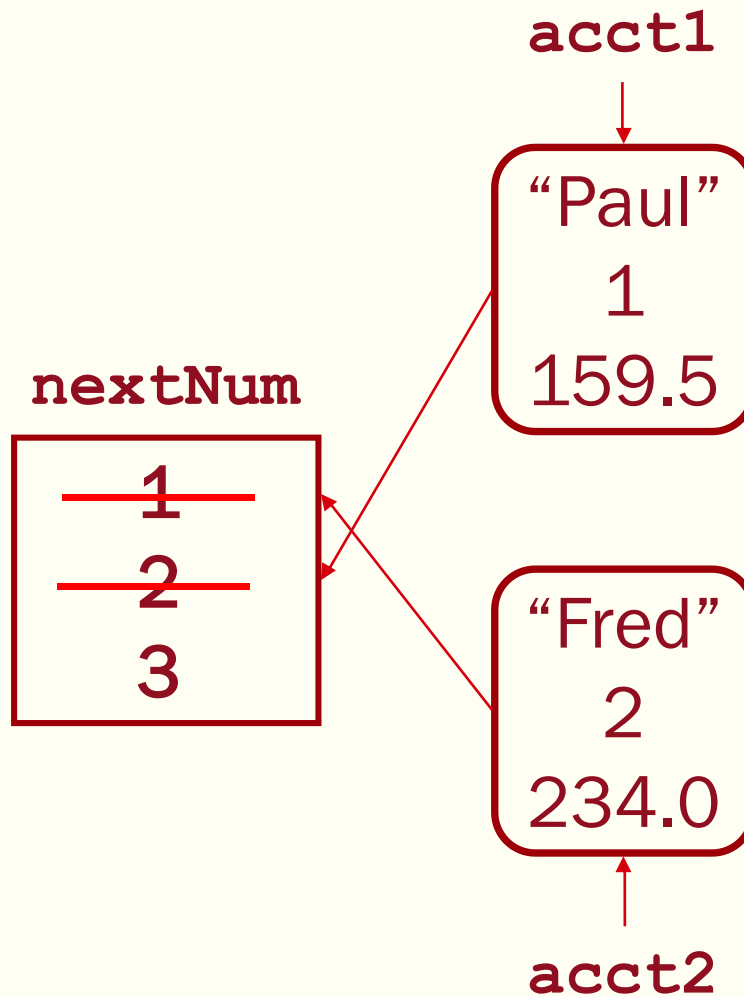
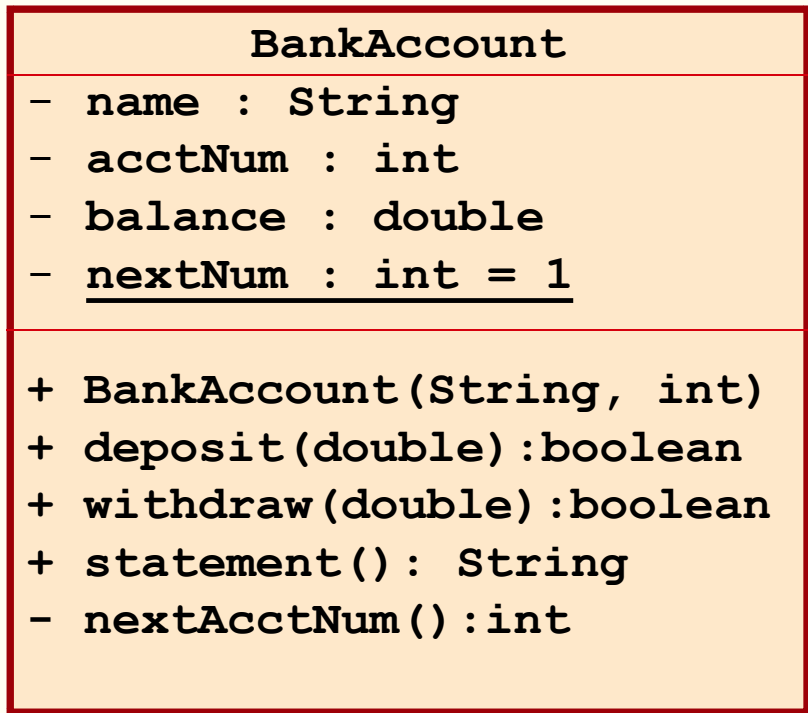
BankAccount
- name : String - acctNum : int - balance : double - <u>nextNum : int = 1</u>
+ BankAccount(String, int) + deposit(double):boolean + withdraw(double):boolean + statement(): String - nextAcctNum():int

```
public BankAccount(String name) {  
    this.name = name;  
    this.acctNum = nextAcctNum();  
    this.balance = 0.0;  
}
```

```
private static int nextAcctNum(){  
    int num = nextNum;  
    nextNum++;  
    return num;  
}
```

Class data & methods in UML are underlined.

Class Variables & Class Methods (5)



Class variables – a single copy available across all instances of a class

Class methods – defined as static – accessed via class name

Underlined in UML

Class Variables & Class Methods (6)

BankAccount

- name : String
- acctNum : int
- balance : double
- nextNum : int = 1

- + BankAccount(String, int)
- + deposit(double):boolean
- + withdraw(double):boolean
- + statement(): String
- nextAcctNum():int

How could we modify the **BankAccount** class to keep track of the *total amount of money* on deposit within the bank?

BankAccount

- name : String
- acctNum : int
- balance : double
- nextNum : int = 1
- totalAmt : double = 0.0

- + BankAccount(String, int)
- + deposit(double):boolean
- + withdraw(double):boolean
- + statement(): String
- nextAcctNum():int
- +audit():String

Class Variables & Class Methods (7)

BankAccount

- name : String
- acctNum : int
- balance : double
- nextNum : int = 1
- totalAmt : double = 0.0

- + BankAccount(String, int)
- + deposit(double):boolean
- + withdraw(double):boolean
- + statement(): String
- nextAcctNum():int
- +audit():String

```
public BankAccount(String name) {
    this.name = name;
    this.acctNum = nextAcctNum();
    this.balance = 0.0;
}

public boolean deposit(double amt){
    if ( amt > 0.0 ){
        balance += amt;
        totalAmt += amt;
        return true;
    }
    return false;
}
```

Class Variables & Class Methods (8)

BankAccount

- name : String
- acctNum : int
- balance : double
- nextNum : int = 1
- totalAmt : double = 0.0

- + BankAccount(String, int)
- + deposit(double):boolean
- + withdraw(double):boolean
- + statement(): String
- nextAcctNum():int
- +audit():String

```
public boolean withdraw(double amt){
    if (amt>0.0 && (balance-amt)>=0.0){
        balance -= amt;
        totalAmt -= amt;
        return true;
    }
    return false;
}

public static String audit() {
    String res="Bank funds:"+totalAmt;
    return res;
}
```

Class Variables & Class Methods (9)

BankAccount

```
- name : String
- acctNum : int
- balance : double
- nextNum : int = 1
- totalAmt : double = 0.0

+ BankAccount(String, int)
+ deposit(double):boolean
+ withdraw(double):boolean
+ statement(): String
- nextAcctNum():int
+audit():String
```

```
BankAccount acct1, acct2, acct3;
acct1 = new BankAccount("Homer");
acct2 = new BankAccount("Marge");
acct3 = new BankAccount("Bart");
acct1.deposit(20);
acct2.deposit(30);
acct3.deposit(40);
String info = BankAccount.audit();
System.out.println(info);
```

BankAccount.java
BankAccountV2.java
BankAccountV3.java
TestBank.java

Enumerated Types (1)

What options are available for representing values such as days of the week, colours, playing cards in a pack, months of the year?

You could perhaps consider using integers to represent the different days of the week (for example):

```
int sun=1, mon=2, tue=3, wed=4, thur=5, fri=6, sat=7;  
int today = mon;
```

What are the issues with this approach? What would be the meaning of the following assignment?

```
today = 8;
```

Enumerated Types (2)

Enumerations are useful for representing a constant range of values.

```
enum Day {  
    Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday;  
}
```

This definition can be used in the following way:

```
Day today = Day.Monday;
```

So **today** may only be assigned from a restricted range of values and can't be assigned outside this range.

Enumerated Types (3)

```
enum Day {  
    Sunday(1), Monday(2),  
    Tuesday(3), Wednesday(4),  
    Thursday(5), Friday(6),  
    Saturday(7);  
  
    private int dayValue;  
  
    private Day(int value) {  
        dayValue = value;  
    }  
  
    public int getValue() {  
        return dayValue;  
    }  
}
```

Enumerations (like classes) may include variables and methods, including constructors.

Can be used in the following way:

```
Day today = Day.Monday;  
int value = today.getValue();  
System.out.println("Value:"+value);
```

Enumerated Types (4)

```
enum Day {  
    Sunday(1), Monday(2), Tuesday(3), Wednesday(4), Thursday(5), Friday(6),  
                                                Saturday(7);  
    private String dayNames[] = {"Sunday", "Monday", "Tuesday", "Wednesday",  
                                "Thursday", "Friday", "Saturday"};  
  
    private int dayValue;  
    private Day(int value) {  
        dayValue = value;  
    }  
    public int getValue() {  
        return dayValue;  
    }  
    public String getName() {  
        return dayNames[dayValue-1];  
    }  
}
```

```
Day today = Day.Monday;  
int value = today.getValue();  
System.out.println("Value:"+value);  
String name = today.getName();  
System.out.println("Name:"+name);
```



What's Next?

Code Documentation

Naming Conventions

Design Guidelines

